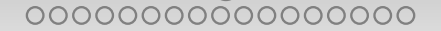


Logica di Base

Mario Bravetti
`mario.bravetti@unibo.it`

Ingegneria e Scienze Informatiche, Cesena
ALMA MATER STUDIORUM—Università di Bologna





Introduction

Propositional Logic

Deduction Systems

Predicate Logic



Goals

Understanding:

- the general framework of logic
- propositional and predicate (first-order) logics
- deductive systems

Later in this course we will see:

- the role of logics in computational models
- what is model checking



What is logic and why using it

Teoria fondamentale che sta alla base di gran parte della matematica e dell'informatica.

It is a foundational theory

La logica è una meta-teoria perché studia le proprietà di altre teorie, analizzandone la struttura, la sintassi e la semantica. In questo senso, la logica non si occupa di un argomento specifico, ma dei principi e delle regole formali che governano il ragionamento corretto in qualsiasi teoria.

Costruita sulla Teoria degli Insiemi

- Built on top of set theory

La Logica e la Teoria degli Insiemi sono strettamente interconnesse. Molti concetti logici hanno un'analogia diretta nelle operazioni tra insiemi.

Può essere usata per definire la Teoria degli Insiemi

- Can itself be used to define set theory

La logica è il metalinguaggio che definisce le regole di base per costruire tutta la matematica, compresa la Teoria degli Insiemi.

What it is used for

Una Proposizione è una frase dichiarativa che può essere solamente Vera o Falsa. I Connettivi Logici definiscono le relazioni tra queste proposizioni, permettendoci di costruire formule complesse e di calcolarne il valore di verità.

La Logica esprime Proposizioni e Relazioni tra esse

- A logic expresses propositions and relations between them

- (deep relation with philosophy)

La logica formale è nata dalla filosofia, in particolare con Aristotele. Si occupa dell'analisi della struttura del ragionamento corretto e della validità delle argomentazioni, che sono questioni filosofiche fondamentali.

In Computer Science

La logica è usata per specificare esattamente cosa deve fare un sistema:

- Verifica di Programmi: Si usano formule logiche per esprimere la preconditione (cosa deve essere vero prima di eseguire un codice) e la postcondizione (cosa sarà vero dopo l'esecuzione del codice).
- Specifiche Formali: Nei sistemi critici (software di controllo aereo, centrali nucleari), la logica è usata per specificare senza ambiguità tutti i requisiti.

Fornisce un Linguaggio per Esprimere le Proprietà di un Sistema

- Provides a language for expressing properties of interest of a system

Fornisce un Modo per Attuare queste Proprietà nel Sistema

- Provides a way of enacting these properties into the system

La logica si traduce direttamente nella struttura fisica e operativa del computer:

- Circuiti Digitali/Porte Logiche: Alla base di ogni microprocessore ci sono le Porte Logiche (AND, OR, NOT), che sono la realizzazione fisica dei connettivi logici. Il valore Vero è l'alta tensione, Falso è la bassa tensione.
- Programmazione Logica: Linguaggi come Prolog utilizzano la logica dei predicati come base. Il programma è una collezione di fatti e regole logiche, e il computer usa l'inferenza logica per rispondere alle query.
- Database: La logica è fondamentale per le interrogazioni e la gestione dei dati.



Basic elements of a logic

Syntax

La Sintassi definisce come scrivere una formula che sia corretta (ben formata) all'interno del sistema logico. Riguarda solo la forma e le regole di costruzione, ignorando il significato.

A set of admissible *formulas* $\mathcal{F} = \{\phi_0, \phi_1, \dots\}$

Sono l'insieme di tutte le espressioni che seguono le regole grammaticali della logica.

- Typically defined by a context-free grammar

Semantics

La Semantica definisce il significato di una formula. Stabilisce come interpretare la formula e se essa è Vera o Falsa.

Defined by two elements $\langle \mathcal{M}, \models \rangle$

- $\mathcal{M}$ is a set of *models* ($M \in \mathcal{M}$)
- $\models \subset \mathcal{M} \times \mathcal{F}$ is the *satisfaction* relation

Un Modello (M) è un'interpretazione o una valutazione che assegna un valore di verità ai simboli di una formula.

Esempio pratico:
Se phi è "Piove" e M1 è un modello dove "Piove = Vero", allora la coppia (M1, phi) fa parte del sottoinsieme $\models$.
Se M2 è un modello dove "Piove = Falso", la coppia (M2, phi) non appartiene a $\models$.

La Relazione di Soddisfazione è la regola che definisce quando un modello M rende vera una formula phi.

Terminology and notation:

- $M \models \phi$ means: model M satisfies formula ϕ
- (or also: formula ϕ is satisfied in interpretation/valuation M)
- Also write $M \models \phi, \phi', \dots$ for: $M \models \phi$ and $M \models \phi'$ and $\dots$

Si legge "M soddisfa phi" oppure "phi è vera nel modello M."

Una notazione abbreviata per dire che lo stesso modello M rende vere contemporaneamente più formule



Propositional Logic

It is the most basic logic introduced

Syntax of a formula ϕ

La Sintassi ci dice quali sequenze di simboli sono formule ben formate e quali no. Definisce la grammatica del nostro linguaggio.

Rappresentano le affermazioni elementari a cui possiamo assegnare un valore di verità (Vero o Falso).

Given a finite set of propositional symbols ($P = \{p, q, r, \dots\}$), a formula can be:

- $\top$ (true constant) Sempre vera, indipendentemente da tutto
- $\perp$ (false constant) Sempre falsa, indipendentemente da tutto.
- a propositional symbol $p \in P$ l'elemento di base
- of the kind $\neg\phi$ (informal meaning: not ϕ) Inverte il valore di verità di phi
- of the kind $\phi \vee \phi'$ (informal meaning: ϕ or ϕ') Vera se almeno una delle due è vera.
- of the kind $\phi \wedge \phi'$ (informal meaning: ϕ and ϕ') Vera solo se entrambe sono vere.
- of the kind $\phi \rightarrow \phi'$ (informal meaning: ϕ implies ϕ') Se phi è vera, allora anche phi' lo è. Spesso letta "Se phi, allora phi"

nothing else can be a formula ϕ .



Propositional Logic

More formally, using context-free grammars.

Per definire la Sintassi della Logica Proposizionale in modo non ambiguo si usa la notazione tipica della Context-Free Grammar.

Syntax

Questa definizione è ricorsiva: le formule complesse sono costruite a partire da formule più semplici.

Grammatica scritta come CFG

Una formula phi può essere

- Grammar: $\phi ::= p \mid \top \mid \perp \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi$
- p is a *proposition* symbol ($p \in P = \{p, q, r, \dots\}$)
- Example: $p \wedge (q \vee r) \rightarrow \neg p$

Model/Interpretation/Valuation

Un modello M associa valori booleani ad ogni simbolo proposizionale

- A model M is a function $M : P \mapsto \{F, T\}$

- It associates a boolean value to each proposition

Il modello M fa l'unica cosa che conta nella logica proposizionale: stabilisce lo stato di verità del mondo che stiamo analizzando.

Modello (M): Non è altro che un modo specifico di assegnare i valori di verità a tutti i simboli proposizionali P.

M è una funzione matematica:

- Dominio (P): L'insieme di tutti i simboli proposizionali ($\{p, q, r, \dots\}$).
- Codominio (F,T): L'insieme dei due valori booleani: Falso (False) e Vero (True).

Il modello mi dice, data una proposizione, i valori per ogni simbolo e quindi il valore della proposizione.



Propositional Logic

Semantics

Truth tables for logical connectives:

ϕ	ψ	and $\phi \wedge \psi$	ϕ	ψ	or $\phi \vee \psi$
T	T	T	T	T	T
T	F	F	T	F	T
F	T	F	F	T	T
F	F	F	F	F	F

ϕ	ψ	implica $\phi \rightarrow \psi$	ϕ	not $\neg \phi$
T	T	T	T	F
T	F	F	F	T
F	T	T		
F	F	T		

Numero di possibili modelli di una formula: (numero di valori di verità)^(numero di simboli proposizionali)

Propositional Logic

Semantics

la Semantica è l'insieme di regole che, dato un Modello (M), stabilisce se una formula phi è Soddisfatta (Vera) in quel modello. La relazione chiave è $M \models \phi$. La definizione è ricorsiva: prima si definiscono le proposizioni atomiche, poi si usano i connettivi per definire le formule complesse.

Definition of satisfaction relation $M \models \phi$

Regole
Base

- $M \models p \iff M(p) = \mathbf{T}$ Il simbolo proposizionale p è vero nel modello M se e solo se la valutazione M le ha assegnato il valore True (Vero).
- $M \models \mathbf{T}$ Questa costante è sempre soddisfatta, in qualsiasi modello M.
- $M \models \perp$ does not hold (written $M \not\models \perp$) Questa costante non è mai soddisfatta (è sempre falsa), in nessun modello M.
- $M \models \neg \phi \iff \text{not } M \models \phi$ (i.e. $M \not\models \phi$) è vera se e solo se phi non è vera nel modello M. (NOT)
- $M \models \phi \wedge \phi' \iff M \models \phi \text{ and } M \models \phi'$ è vera se e solo se entrambe le formule, phi e phi', sono vere. (AND)
- $M \models \phi \vee \phi' \iff M \models \phi \text{ or } M \models \phi'$ è vera se e solo se almeno una tra phi e phi' è vera. (OR)
- $M \models \phi \rightarrow \phi' \iff M \models \phi \text{ implies } M \models \phi'$ è vera in tutti i casi, tranne quando phi (l'antecedente) è vera e phi' (il conseguente) è falsa. (IMPLICA)

that is application of *truth tables* for “not”, “and”, “or”, “implies”.

Example

Modello che ha assegnato False a 'p' e True a 'q'

- $\{p \mapsto F, q \mapsto T\} \models \mathbf{T}, \neg \perp, q, \neg p, q \vee p, q \wedge \neg p, p \rightarrow q$



On Propositional Logic

Scope

la Logica Proposizionale è essenziale perché fissa le regole del gioco: è il sistema più semplice che ci permette di definire il concetto di validità logica e inferenza usando un linguaggio formale e non ambiguo.

La Logica Proposizionale è il punto di partenza per tutti i sistemi logici più complessi

- It is the basic logic upon which all others are built
- It is the framework where standard concepts are formally defined (true, false, or, and, ...)

La Logica Proposizionale traduce il linguaggio naturale ambiguo in termini matematici rigorosi, stabilendo una volta per tutte cosa significano concetti come "vero", "falso" e "implica"



Some general concepts about logics (1)

Logical Consequence

La Conseguenza Logica ci permette di stabilire se una conclusione (ϕ) segue necessariamente da un insieme di premesse

- Definition: Formula ϕ is *logical consequence* of $\phi_1, \dots, \phi_n$ if,
 per qualsiasi Modello
 for any M , when $M \models \phi_1, \dots, \phi_n$ then also $M \models \phi$

- This can be expressed by notation: $\phi_1, \dots, \phi_n \models \phi$

- Examples in Propositional Logic: $p, q \models p \wedge q, p \vee q$
 $p \wedge q \models p \vee q$

Diciamo che ϕ è conseguenza logica di $\{\phi_1, \dots, \phi_n\}$ se, in tutti i modelli in cui tutte le premesse $\{\phi_1, \dots, \phi_n\}$ sono vere, anche ϕ deve essere necessariamente vera. Questo significa che in tutti i modelli M in cui $\{\phi_1, \dots, \phi_n\}$ è vera, negli stessi modelli M anche ϕ è vera

La formula ϕ è conseguenza logica dell'insieme di formule $\{\phi_1, \dots, \phi_n\}$ se, per qualsiasi Modello M , nel momento in cui tutte le premesse sono vere ($M \models \phi_1, \dots, \phi_n$), allora anche la conclusione ϕ è vera ($M \models \phi$).
- Significato Chiave: L'esistenza di un modello M che rende vere tutte le premesse e falsa la conclusione (ϕ) è sufficiente a negare la relazione di conseguenza logica.
- Terminologia: L'insieme $\{\phi_1, \dots, \phi_n\}$ è chiamato l'ipotesi o il supporto logico.

Tautology/Validity

Una Tautologia è l'esempio più forte di conseguenza logica, in cui la formula è vera senza che ci sia bisogno di alcuna premessa.

- Definition: A formula ϕ that is logical consequence of no formulas (i.e. it is always true in any model) is called a *tautology*

Una tautologia è una formula che è vera in ogni possibile modello, senza bisogno di alcuna premessa. La formula è vera "per struttura". Non esiste alcun mondo (interpretazione) che possa renderla falsa.

- Notation: $\models \phi$ (we also say that ϕ is *valid*)

- Examples in Propositional Logic: $\models \top, \neg \perp, p \rightarrow p, p \vee \neg p$

Indipendentemente dal modello M , ho soddisfacimento della proposizione



Some general concepts about logics (2)

More complex examples of logical consequence

alfa e beta proposizioni

- $\alpha \models \beta \rightarrow \alpha$ Se l'affermazione alpha è Vera (la nostra premessa), allora l'implicazione beta $\rightarrow$ alpha è sempre Vera, indipendentemente dal valore di verità di beta.
- $\alpha \rightarrow (\beta \rightarrow \gamma) \models (\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma)$
- $\neg\alpha \rightarrow \neg\beta \models \beta \rightarrow \alpha$ Questa è una delle equivalenze più importanti. Dice che l'affermazione "Se non alpha, allora non beta" è logicamente equivalente a "Se beta, allora alpha."

Proof by contradiction Dimostrazione per Assurdo

If $\phi_1, \dots, \phi_n, \neg\phi \models \perp$ then $\phi_1, \dots, \phi_n \models \phi$

Vogliamo dimostrare che l'affermazione phi è conseguenza logica delle premesse $\phi_1, \dots, \phi_n$.

- Procedura:

> Assumiamo le premesse come vere: $\phi_1, \dots, \phi_n$.

> Assumiamo la negazione della conclusione che vogliamo dimostrare come vera: not phi.

> Se da queste ipotesi congiunte ($\phi_1, \dots, \phi_n, \text{not phi}$) riusciamo a derivare una contraddizione (il simbolo del Falso Assoluto), significa che l'assunzione not phi era assurda.

> Pertanto, se not phi porta ad un assurdo, la conclusione originale phi deve essere vera.

An exercise

Qual è la conseguenza logica di premesse contraddittorie?

What is the logical consequence of contradictory premises?

- $\phi, \neg\phi \models \alpha$, which α ? alfa può essere qualsiasi cosa (essendo che anche un'implicazione, alpha è true se a sinistra del soddisfa è true, alpha è qualsiasi cosa se a sinistra del soddisfa è false, essendo un'implicazione) questo perché la prima parte non mi dice nulla sulla seconda se la prima è falsa
- that is, for any M it must hold:

Per ogni Modello M deve valere:

whenever $M \models \phi, \neg\phi$ then also $M \models \alpha$

allora

Analizziamo l'antecedente: ' $M \models \phi$ ' e ' $M \models \text{not phi}$ '.
- Per la semantica della negazione, l'affermazione ' $M \models \text{not phi}$ ' è vera se e solo se M non soddisfa phi.
- Quindi, l'antecedente richiede che: ' $M \models \phi$ ' e ' M non soddisfa phi'.
Questo è impossibile. Non esiste alcun modello M in cui sia phi che not phi siano vere contemporaneamente. L'insieme di premesse (phi, not phi) è inconsistente (o insoddisfacibile).

Conclusione Logica (Principio di Esplosione)
Poiché l'antecedente (la condizione Quando ' $M \models \phi$ ' e ' $M \models \text{not phi}$ ') non è mai verificato in alcun modello, l'intera implicazione logica è vacuamente vera per qualsiasi conclusione alpha.
phi, not phi soddisfa alpha dove alpha è una formula qualsiasi.
Risposta: La conseguenza logica di premesse contraddittorie è qualsiasi formula alpha.



Some general concepts about logics (3)

Logical Equivalence

L'Equivalenza Logica definisce quando due formule, pur avendo una sintassi diversa (sono scritte in modo diverso), hanno esattamente lo stesso significato semantico (lo stesso valore di verità in ogni possibile modello).

Le formule ϕ e ϕ' sono logicamente equivalenti se ϕ è conseguenza logica di ϕ' e ϕ' è conseguenza logica di ϕ .

- Definition: Formula ϕ and ϕ' are said *logically equivalent* (written $\phi \equiv \phi'$) if $\phi \models \phi'$ and $\phi' \models \phi$

Due formule equivalenti possono essere usate intercambiabilmente in qualsiasi contesto logico senza cambiare il risultato finale. Questo è fondamentale per l'ottimizzazione e la trasformazione delle formule.

Examples in Propositional Logic

- De Morgan Law: $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$

La negazione di una disgiunzione (OR) è equivalente alla congiunzione (AND) delle negazioni.

Formule Equivalenti dell'Implicazione

- $p \rightarrow q \equiv (\neg p) \vee q \equiv (\neg q) \rightarrow (\neg p)$



Some general concepts about logics (4)

Satisfiability

Una formula ϕ è soddisfacibile se esiste almeno un Modello M (una singola assegnazione di valori Vero/Falso) che rende la formula Vera.
Significato Chiave: Per essere soddisfacibile, non è necessario che la formula sia sempre vera per ogni modello (come per la Validità), ma basta che non sia sempre falsa.
Contrario: Una formula che non è soddisfacibile è detta insoddisfacibile o inconsistente.

- Definition: Formula ϕ is *satisfiable* if $M \models \phi$ for some M
- Examples in Propositional Logic:
 - $\top, p, p \vee q$ are satisfiable
 - $\neg\top, \perp, p \wedge \neg p$ are *not* satisfiable.

Relazione tra Soddisfacibilità e Validità

On relationship between satisfiability and validity

ϕ is valid if and only if $\neg\phi$ is not satisfiable

$\models \phi$ (Significa che ϕ è Vera in ogni Modello M)

$\iff$

non esiste nessun Modello M che possa rendere $\neg\phi$ Vera.

Se $\neg\phi$ non può mai essere Vera, significa che ϕ deve essere sempre Vera (poiché in ogni M o è vera ϕ o è vera $\neg\phi$).

Questa equivalenza è essenziale per l'Informatica. Per provare che una formula ϕ è Valida (cioè che è un teorema), l'algoritmo non cerca di controllare tutti gli infiniti modelli. Invece, cerca di dimostrare che la sua negazione ($\neg\phi$) è Insoddisfacibile.



Some general concepts about logics (5)

La definizione di $\phi \models \psi$ dice: "In ogni modello in cui ϕ è vera, anche ψ deve essere vera". Ora guarda la tabella di verità dell'implicazione $\phi \rightarrow \psi$: L'unico caso in cui $\phi \rightarrow \psi$ è falsa è quando ϕ è vera e ψ è falsa. Se ψ è conseguenza logica di ϕ , quel caso "proibito" (Vero $\rightarrow$ Falso) non può mai verificarsi in nessun modello. Di conseguenza, la formula $\phi \rightarrow \psi$ risulterà sempre vera in tutti i modelli possibili. Una formula sempre vera è, per definizione, una tautologia.

On Logical Consequence

Questa regola ci dice che l'affermazione "la conclusione ϕ deriva dalle premesse" è equivalente all'affermazione "una singola formula di implicazione è una tautologia."

$$\phi_1, \dots, \phi_n \models \phi \text{ if and only if } \models (\phi_1 \wedge \dots \wedge \phi_n) \rightarrow \phi$$

Affermare che la conseguenza logica sussiste (LHS) è esattamente equivalente ad affermare che l'unico modello che renderebbe l'implicazione Falsa (premesse Vere e conclusione Falsa) non esiste. Dunque, l'implicazione (RHS) è una Tautologia.

On Logical Equivalence (LE)

Questa regola ci permette di esprimere l'equivalenza logica attraverso una singola formula valida che usa l'operatore di doppia implicazione

$$\phi \equiv \phi' \text{ if and only if } \models (\phi \rightarrow \phi') \wedge (\phi' \rightarrow \phi)$$

- Operator $\leftrightarrow$ is sometimes added to mean LE ($\dots \models \phi \leftrightarrow \phi'$)

A note

Questi fatti non valgono in generale, dato che non tutte le logiche possono avere l'operatore 'implica', tuttavia valgono in tutte le logiche che estendono la Logica Proposizionale, cioè su tutte le logiche standard.

These facts do not hold in general, since not all logics may have operator $\rightarrow$, however they hold in all logics extending Propositional Logic, that is, on all standard logics

Importanza per l'Informatica: Poiché la stragrande maggioranza dei sistemi logici usati in Informatica estendono la Logica Proposizionale, queste relazioni di riduzione valgono sempre.



A simplified version of Propositional Logic

Il concetto fondamentale è che non abbiamo bisogno di tutti i connettivi logici per esprimere ogni possibile relazione logica. Un sottoinsieme minimo di operatori, chiamato set funzionalmente completo, è sufficiente.

Some equivalence

All operators can be defined in terms of $\neg$ and $\rightarrow$ only:

$$\begin{aligned}\top &\equiv p \rightarrow p \\ \perp &\equiv \neg(p \rightarrow p) \\ \phi \vee \phi' &\equiv (\neg\phi) \rightarrow \phi' \\ \phi \wedge \phi' &\equiv \neg(\phi \rightarrow \neg\phi')\end{aligned}$$

Syntax

$$\phi ::= p \mid \neg\phi \mid \phi \rightarrow \phi$$

(Semantics is the simplified version of what we saw before)

(BTW: Another simplified syntax is $\phi ::= p \mid \perp \mid \phi \rightarrow \phi$)



A problem with validity

In PL, non abbiamo questo problema: come abbiamo visto, la Validità in PL è decidibile grazie al metodo delle Tavole di Verità. Poiché ci sono 2^n modelli (finiti) per n variabili, possiamo sempre completare il calcolo in un tempo finito.

Motivation

Abbiamo bisogno di un metodo per verificare la validità di una formula ($\models \phi$), ma non sempre la definizione di Semantica (la relazione $\models$) ci offre uno strumento di calcolo pratico.

Given a logic $\langle \mathcal{F}, \models \rangle$:

- Suppose we want to check whether $\phi \in \mathcal{F}$ is valid, that is $\models \phi$
- Suppose the definition of relation $\models$ does not suggest an ^{efficace} effective way of ^{calcolare} computing it (for Propositional Logic we do not have this problem, for other Logics we do have it)
- We may need a procedure to check validity, ^{ad esempio} e.g., using a ^{simile alla} mechanism resembling simplification of mathematical expressions (a *calculus*) or theorem proving in mathematics

Potremmo aver bisogno di una procedura per verificare la validità, ad esempio, utilizzando un meccanismo che assomigli alla semplificazione di espressioni matematiche (un calcolo) o alla dimostrazione di teoremi in matematica. Poiché il metodo semantico è impraticabile (o impossibile) in logiche complesse, l'unica via è creare un Sistema di Deduzione (Proof System) che sia puramente sintattico.

Un Sistema di Deduzione è un insieme di regole formali che ci permette di manipolare le formule (sintassi) per dedurre di nuove, senza mai fare riferimento al loro significato (semantica).

- Obiettivo: Trasformare il problema della Validità ($\models \phi$) in un problema di Dimostrabilità ($\vdash \phi$).

- Meccanismo:

> Assiomi: Un piccolo insieme di formule che sono assunte come vere (tautologie).

> Regole di Inferenza: Un piccolo insieme di regole di manipolazione che permettono di passare da formule vere a nuove formule vere.

- Vantaggio: La dimostrazione è una sequenza finita e verificabile di passi che applicano queste regole, partendo dagli assiomi e arrivando alla formula ϕ . Questo processo è completamente algoritmico e non richiede il controllo di modelli infiniti.

In sintesi, i Sistemi di Deduzione sono la risposta al "problema della validità" in logiche complesse, sostituendo la verifica semantica infinita con una dimostrazione sintattica finita.



An example in Propositional Logic

mostra il limite pratico del metodo delle Tavole di Verità, anche per la Logica Proposizionale, e introduce la necessità di tecniche più efficienti, come la Forma Normale Congiuntiva (CNF).

Even in Propositional Logic we can make an example

- Suppose we want to check
 $\models \neg((a \wedge b \wedge c \wedge d \wedge e) \rightarrow c) \rightarrow (f \vee g \vee h \vee i)$
- It amounts to build a tableau of $2^9 = 512$ rows (it could be worse than that)

Resorting to CNF

Per aggirare la complessità esponenziale della Tavola di Verità, si ricorre a forme standard e tecniche di semplificazione. Una delle più importanti è la Forma Normale Congiuntiva (CNF).

A more efficient technique is to turn the formula into CNF

(Conjunctive Normal Form):

- a conjunction ($\wedge$) of clauses
- a clause is a disjunction ($\vee$) of literals
- a literal is a (positive literal) or $\neg a$ (negative literal) for some a

Puoi immaginare la CNF come una struttura a "matrioska":

Letterale: È l'elemento più piccolo. Può essere una variabile atomica (a) o la sua negazione (neg a).

Clausola: È un insieme di letterali legati tra loro solo dall'operatore OR.

Formula CNF: È un insieme di clausole legate tra loro solo dall'operatore AND.

Un letterale è un simbolo proposizionale (a) o la sua negazione (not a).



We just check that each clause contains a pair of opposite literals.

Per testare la Validità di una formula, verifichiamo semplicemente che ogni clausola contenga una coppia di letterali opposti.

Deduction Systems

L'obiettivo è dimostrare che una formula è valida o che una conseguenza logica sussiste, non provando tutti i modelli, ma manipolando le formule usando regole di inferenza finite.

Validity can be “deduced” by the following proof-like reasoning

- 1 → $(a \wedge b \wedge c \wedge d \wedge e) \rightarrow c$ is true since on left we have a conjunction including the implied formula
Se abbiamo una congiunzione che include il conseguente c (ovvero, c è una delle premesse), l'implicazione è sempre vera.
- 2 → $\neg((a \wedge b \wedge c \wedge d \wedge e) \rightarrow c)$ is false
Se l'implicazione interna è sempre vera, la sua negazione è sempre falsa (assioma).
- 3 → a false formula implies any formula
Poiché l'antecedente della formula principale è Falso, l'intera implicazione è Vera (assioma) perché una formula è valida se e solo se la negazione della formula non è soddisfacibile.

Tautologie
(sempre vere)

We used “axioms”: $\phi \wedge \phi' \rightarrow \phi$, $\neg \neg \top$, $\perp \rightarrow \phi$

The notion of Logical Deduction

Il concetto di Deduzione Logica formalizza il ragionamento basato su assiomi e regole.

A *deduction* relation $\vdash$ is introduced:

- $\phi_1, \dots, \phi_n \vdash \phi$ means ϕ can be deduced from $\phi_1, \dots, \phi_n$
può essere dedotta/dimostrata
- in particular $\vdash \phi$ means ϕ can be deduced from no formula
(da un insieme vuoto di premesse)
- ($\models$ is called semantic entailment, $\vdash$ syntactic entailment)

> $\models$ è chiamato Implicazione Semantica (Semantic Entailment): Riguarda il significato, la verità nel mondo (modello).
> $\vdash$ è chiamato Implicazione Sintattica (Syntactic Entailment): Riguarda la forma, la dimostrabilità tramite regole.

Relazione: Conseguenza Logica
Natura: Semantica
Come si Verifica? Controllo su tutti i Modelli (M).

Relazione: Deduzione Logica
Natura: Sintattica
Come si Verifica? Costruzione di una Prova (sequenza di regole).



Axioms and rules

Defining a deduction system

Mentre la relazione di conseguenza logica ($\models$) viene definita semanticamente (controllando i modelli), la relazione di deduzione ($\vdash$) viene sempre definita costruttivamente, attraverso l'uso di assiomi e regole. Una dimostrazione in un sistema deduttivo è una sequenza finita di applicazioni di queste regole e assiomi.

While relation $\models$ can be defined in any mathematical way, relation $\vdash$ is always defined constructively, by *axioms* and *rules*

- Form of axioms: $\vdash \phi$ Gli assiomi sono i punti di partenza, le verità elementari che non richiedono prova all'interno del sistema. Puoi pensare a un assioma come a una regola di inferenza che non ha precondizioni (ipotesi zero).

- Form of rules:
$$\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$$
 Le regole di inferenza specificano come dedurre una nuova formula (la conclusione) da un insieme di formule già dimostrate (le premesse).

(it means: ϕ holds if all preconditions $\phi_1, \dots, \phi_n$ hold)

where $\phi, \phi_1, \dots$ are formulas or formula schemas (formulas with variables, that can be substituted by any formula)

Schemi di Formule: Gli assiomi e le regole non si applicano a specifiche proposizioni ($\$p, q\$$) ma a schemi generici.
Variabili Metalinguistiche: I simboli come phi sono variabili che stanno per qualunque formula ben formata della Logica Proposizionale (o della logica in uso).



Questo sistema è notevole perché raggiunge la Completezza (può dimostrare ogni Tautologia) con un numero estremamente ridotto di assiomi e una sola regola di inferenza.

Hilbert deduction system

il sistema più minimalista e fondamentale per dimostrare la validità in Logica Proposizionale

Deduction system

Minimal syntax: $\phi ::= p \mid \neg\phi \mid \phi \rightarrow \phi$

Axioms:

- $\vdash \alpha \rightarrow (\beta \rightarrow \alpha)$
- $\vdash (\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$
- $\vdash (\neg\alpha \rightarrow \neg\beta) \rightarrow (\beta \rightarrow \alpha)$

Rule:

- $$\frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$

Note: parenthesis are not part of the language, they are just a way to group sub-expressions

La Sintassi (la grammatica) della formula non include le parentesi. Tuttavia, sono usate come simboli ausiliari per eliminare l'ambiguità e specificare l'ordine di valutazione delle sotto-espressioni (ad esempio, distinguere $\alpha \rightarrow (\beta \rightarrow \gamma)$ da $(\alpha \rightarrow \beta) \rightarrow \gamma$). Senza di esse, la formula sarebbe mal formata o ambigua.

Derivation of a deduction

Derivation of a deduction

Proving $\phi_1 \dots \phi_n \vdash \phi$ is done by proving $\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$, that is by deriving $\vdash \phi$ under the assumptions $\vdash \phi_1 \dots \vdash \phi_n$

Deduction system

$$\text{A1} \vdash \alpha \rightarrow (\beta \rightarrow \alpha)$$

$$\text{A2} \vdash (\alpha \rightarrow (\beta \rightarrow \gamma)) \rightarrow ((\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \gamma))$$

$$\text{A3} \vdash (\neg \alpha \rightarrow \neg \beta) \rightarrow (\beta \rightarrow \alpha)$$

$$\text{MP} \frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$

La derivazione di Hilbert è puramente sintattica e si basa sull'identificazione di schemi. L'efficacia di questa dimostrazione risiede nel fatto che non abbiamo mai fatto riferimento al significato di alpha, beta, gamma.

We prove $\alpha \rightarrow \gamma, \gamma \rightarrow \beta \vdash \alpha \rightarrow \beta$ that is $\frac{\vdash \alpha \rightarrow \gamma \quad \vdash \gamma \rightarrow \beta}{\vdash \alpha \rightarrow \beta}$

L'esempio mostrato cerca di dimostrare la regola della Transitività dell'Implicazione (o Sillogismo Ipotesico)

$$\begin{array}{c} \vdash \alpha \rightarrow \gamma \quad [MP] \quad \frac{\vdash \gamma \rightarrow \beta \quad [A1] \quad \frac{\vdash (\gamma \rightarrow \beta) \rightarrow (\alpha \rightarrow (\gamma \rightarrow \beta))}{\vdash \alpha \rightarrow (\gamma \rightarrow \beta)}}{\vdash (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta)} \quad [A2] \quad \frac{\vdash (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta)}{\vdash \alpha \rightarrow \beta} \\ [MP] \frac{\vdash (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta) \quad \vdash \alpha \rightarrow \gamma}{\vdash \alpha \rightarrow \beta} \end{array}$$



Soundness and completeness of Hilbert deduction system

Queste due proprietà, Correttezza (Soundness) e Completezza (Completeness), non sono solo teoremi, ma sono i pilastri che garantiscono l'affidabilità e la potenza di un sistema logico come quello di Hilbert.

Essi stabiliscono che la verità che dimostriamo sulla carta (Sintassi) corrisponde perfettamente alla verità nel mondo dei modelli (Semantica).

La Correttezza garantisce che il sistema di deduzione sia affidabile e non possa produrre "verità" errate.

Soundness of $\vdash$ with respect to $\models$

Se una formula ϕ può essere dimostrata (dedotta) dalle premesse $\phi_1, \dots, \phi_n$ usando gli assiomi e le regole (come il Modus Ponens), allora essa è anche una conseguenza logica di quelle premesse.

Significato: Questo significa che tutto ciò che dimostri è vero (nel senso semantico). Il sistema di Hilbert non ti farà mai dedurre una conclusione falsa da premesse vere.

If $\phi_1, \dots, \phi_n \vdash \phi$ then $\phi_1, \dots, \phi_n \models \phi$

That is: Le deduzioni derivate sono effettivamente conseguenze logiche! derived deductions are indeed logical consequences!

La Completezza garantisce che il sistema di deduzione sia abbastanza potente da dimostrare tutte le verità logiche possibili.

Completeness of $\vdash$ with respect to $\models$

Se una formula ϕ è una conseguenza logica delle premesse $\phi_1, \dots, \phi_n$ (cioè, è vera in tutti i modelli che rendono vere le premesse), allora è possibile trovare una dimostrazione formale di ϕ a partire da quelle premesse nel sistema di Hilbert.

Significato: Questo significa che tutte le verità logiche possono essere dimostrate. Non esistono conseguenze logiche che il sistema non sia in grado di cogliere.

If $\phi_1, \dots, \phi_n \models \phi$ then $\phi_1, \dots, \phi_n \vdash \phi$

Tutte le deduzioni che sono conseguenze logiche possono essere derivate!

All deductions that are logical consequences can be derived!



Decidability of Hilbert deduction system

La Decidibilità è una proprietà cruciale che collega la teoria della logica alla teoria della computazione.

Dire che il sistema è decidibile significa che esiste un algoritmo capace di rispondere "Sì" o "No" alla domanda: "Esiste una dimostrazione per questa tesi?".
Se la risposta è "Sì", l'algoritmo non si limita a confermarlo, ma costruisce e restituisce l'intera sequenza di passaggi (la derivazione) che porta dalle premesse alla conclusione.

Decidability of Hilbert deduction system

There is an algorithm that establishes whether a derivation for $\phi_1 \dots \phi_n \vdash \phi$, i.e. for $\frac{\vdash \phi_1 \dots \vdash \phi_n}{\vdash \phi}$, exists (by using axioms and rules and instantiating their variables), and computes such a derivation.

Sebbene l'algoritmo diretto per costruire la derivazione di Hilbert sia complesso, l'esistenza di un algoritmo è garantita dal fatto che possiamo:

- Trasformare la deduzione $\phi_1, \dots, \phi_n \vdash \phi$ nell'equivalente problema di validità: $\models (\phi_1 \text{ AND } \dots \text{ AND } \phi_n) \rightarrow \phi$.
- Utilizzare le Tavole di Verità per determinare se questa formula di implicazione è una Tautologia.

Se è una Tautologia (Sì): Sappiamo che la derivazione $\vdash$ esiste (per la Completezza).

Se non è una Tautologia (NO): Sappiamo che la derivazione $\vdash$ non esiste (per la Correttezza).

La Decidibilità è una proprietà molto potente. Significa che il sistema di logica può essere completamente automatizzato da un computer; un aspetto fondamentale per l'informatica teorica e pratica.



← In sintesi, la Logica è così potente perché separa il linguaggio (sintassi) dal significato (semantica). I Sistemi di Deduzione si concentrano sulla sintassi, offrendo un modo algoritmico e oggettivo per dimostrare la validità che è garantito essere affidabile (grazie alla Correttezza e Completezza rispetto alla Semantica).

Deduction systems are pure-syntax!

Offrono un Modo "Tipografico" per Controllare la Validità

- Tipografico: Si riferisce alla manipolazione di simboli e caratteri. Una dimostrazione è una sequenza di formule e di simboli che vengono manipolati e riorganizzati secondo regole precise.
- Non Semantico: Non si verifica la verità in un modello (M); si verifica la correttezza della sequenza di passi sintattici.
- Esempio: L'applicazione del Modus Ponens è un'operazione tipografica. Non devi sapere cosa significano alpha o beta; devi solo riconoscere la forma del pattern e applicare la regola.

The "philosophy" of deduction systems..

- They give a "typographical" way of checking validity of a formula

Non Hai Bisogno di Conoscere il Significato

- You do not need to know the meaning of $\neg$, $\rightarrow$, ...

- Those axioms and rules tell you what is such meaning

Il sistema funziona indipendentemente dal fatto che l'utente interpreti not come "non" o come qualsiasi altro operatore con la tavola di verità della negazione.

Gli Assiomi e le Regole Definiscono il Significato: In un sistema puramente sintattico, gli assiomi e le regole di inferenza definiscono implicitamente il comportamento dei connettivi.



PL non può analizzare la struttura interna delle proposizioni. Non può esprimere concetti come "tutti," "esiste," o proprietà degli oggetti.
- Esempio: Formalizzare "Marco è un uomo" e "Marco è mortale" richiederebbe due proposizioni atomiche separate (p_1 , p_2). PL non vedrebbe alcun collegamento tra i due, anche se logicamente sono correlati.

Predicate Logic

Propositional Logic

La Logica Proposizionale tratta le affermazioni come scatole nere atomiche a cui assegnare un valore di verità (Vero o Falso), senza mai guardare al loro contenuto.

You can model sentences like:

- If it rains, then it is true that it rains or it snows!
- $p = \text{it rains}$, $q = \text{it snows}$
- $\models p \rightarrow p \vee q$

La Logica Proposizionale è sufficiente perché si concentra sulla relazione tra le intere proposizioni (p, q) tramite i connettivi (implica, or).

Predicate (First-Order) Logic

La Logica del Predicato supera questo limite introducendo un vocabolario più ricco per descrivere oggetti, proprietà e relazioni.

Predicates should be more structured, and discuss individual and global properties

- All men are mortal, Socrates is a men, therefore Socrates is mortal
- ...



Predicate Logic

Premesse:

- $man(X)$, $mort(X)$: Predicati (proprietà degli oggetti).
- $socrat$: Costante (un oggetto specifico).
- per ogni X : Quantificatore Universale ("Per ogni $X...$ ").

Significato: "Per ogni individuo X , se X è un uomo, allora X è mortale" E "Socrate è un uomo."

Example 1

Questo esempio è un sillogismo (un tipo di inferenza deduttiva) che coinvolge l'applicazione di una regola generale (tutti gli uomini) a un caso specifico (Socrate).

La formula intera
è una Tautologia
(è valida).

- All men are mortal, Socrates is a men.
Therefore Socrates is mortal

La formalizzazione
corretta del fatto
che la conclusione è
conseguenza logica
delle premesse è:

$$\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$$

Premesse

Example 2

Questo esempio mostra come FOL gestisce le relazioni tra due insiemi di oggetti (studenti e istruttori) e l'uso combinato dei quantificatori.

- Every student is younger than some instructor
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y))))$

$younger(X, Y)$: Predicato binario
(relazione tra due oggetti).
Traduzione: "... Y è un istruttore
E X è più giovane di Y ."



Predicate Logic – Ingredients

FOL arricchisce PL introducendo elementi che permettono di analizzare la struttura interna delle affermazioni.

Examples

- $\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y))))$

Ingredients w.r.t. Propositional Logic

Mentre FOL mantiene tutti i connettivi logici di PL (not, and, or, implica, etc.), aggiunge tre componenti fondamentali: Quantificatori, Predicati e Termini.

- Universal ($\forall$) and existential ($\exists$) quantification
- atomic formulas are not just propositional symbols $p, q, \dots$
- but are predicates applied to terms ($\text{man}/1, \text{younger}/2, \dots$)
- terms can be variables (X) or constants ($socrat$)

I quantificatori permettono di esprimere concetti come "tutti" ed "almeno uno", applicando le proprietà a intere collezioni di oggetti.
Quantificazione Universale: Si legge "Per tutti gli X".
Quantificazione Esistenziale: Si legge "Esiste almeno un Y".

In PL, le formule atomiche sono simboli semplici (p, q). In FOL, sono strutture più complesse: Predicati applicati a Termini.

numero di argomenti che accetta.

proprietà singola

relazione tra due oggetti

Predicati: Rappresentano proprietà di un oggetto o relazioni tra più oggetti.

I Termini sono gli elementi che occupano gli argomenti dei Predicati. Essi rappresentano gli oggetti di cui stiamo parlando.
Variabili (X, Y): Simboli generici che possono essere quantificati.
Costanti ($socrat$): Nomi che si riferiscono a un singolo oggetto specifico e immutabile.



A differenza della Logica Proposizionale, dove il Modello si limitava ad assegnare Vero o Falso ai simboli proposizionali, in FOL il Modello deve interpretare gli oggetti e le proprietà/relazioni.

Predicate Logic – Interpretation

Examples

- $\models (\forall X (man(X) \rightarrow mort(X)) \wedge man(socrat)) \rightarrow mort(socrat)$
- $M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y)))))$

How interpreting 1L

In FOL, un Modello (M) deve fornire un significato specifico (un'interpretazione) a tutti i simboli non logici presenti nel linguaggio: Costanti, Funzioni (se presenti) e Predicati.

La validità di questa formula è dovuta esclusivamente alla sua struttura logica. Non importa se *man* significa "uomo", "gatto" o "numero primo"; la forma "Se una proprietà vale per tutti, allora vale per questo specifico oggetto" è sempre vera in FOL.

- Note that first example above is a tautology independently of the meaning of *man/1*, *mort/1*, and constant *socrat*, that is, independently from the model *M*.
- The second example is true only if *M* is properly introduced that defines how *stud/1*, *instr/1* and *younger/2* work.
- *M* should provide an interpretation for predicates and terms!

La verità di questa formula dipende dal mondo che stiamo descrivendo.

- Modello M1 (Vero): Se *M* è l'interpretazione di un'università dove ogni studente è assegnato a un tutor (istruttore) più anziano, allora M1 soddisfa la formula. $M1 \models \phi$.

- Modello M2 (Falso): Se *M* è l'interpretazione di una classe dove c'è uno studente di 40 anni che è il più anziano di tutti, e nessun istruttore è più giovane di lui, allora M2 non soddisfa la formula. M2 non soddisfa ϕ .

Conclusione: Questa formula è Soddisfacibile (vera in alcuni modelli) ma non Valida (non è vera in tutti i modelli). La sua verità è condizionale all'interpretazione *M*.



Syntax

Terms

A term t over (C, F) has syntax: $t ::= x \mid c \mid f(t_1, \dots, t_n)$

- a variable $(x, y, z, \dots)$ Un nome generico, quantificabile. Rappresenta un qualsiasi individuo nel dominio (es. qualsiasi persona).
- a constant $(c, d, e, \dots \in C)$ Un nome specifico che si riferisce a un singolo individuo (es. la persona chiamata Alice).



- a compound term: an n -ary function $(f \in F)$ with t_i terms as arguments Un'espressione che si valuta in un singolo individuo. Una funzione f in F con numero di parametri n , dove $t_1, \dots, t_n$ sono termini.

a term yields an individual in the domain of discourse!

Questa è la Semantica essenziale dei Termini:

- Termini = Oggetti: Quando un termine viene interpretato in un Modello M , il suo risultato è sempre un singolo elemento dell'universo dei valori (D).
- Termini != Frasi: Un Termine non può essere Vero o Falso. Solo le Formule (che includono i Predicati) possono avere un valore di verità.



Syntax

Le formule sono l'unica parte del linguaggio FOL che può essere Vera o Falsa.

Formulas

A formula ϕ over (C, F, P) has syntax:

$\phi ::= p(t_1, \dots, t_n) \mid \top \mid \perp \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid \phi \rightarrow \phi \mid \forall x\phi \mid \exists x\phi$

Un predicato p in P con numero di parametri n , dove $t_1, \dots, t_n$ sono termini.

- an atomic formula: an n -ary predicate ($p \in P$) with t_i terms over (C, F) as arguments (n can be 0)

- a formula $\top, \perp, \neg\phi, \phi \wedge \phi, \phi \vee \phi, \phi \rightarrow \phi$ (as for propositions)

- a universal quantification of x in ϕ

- an existential quantification of x in ϕ

Queste sono le regole che permettono di esprimere la generalità.
Quantificazione Universale: La formula ϕ è vera per tutti i possibili valori che la variabile x può assumere nell'universo dei valori.
Quantificazione Esistenziale: La formula ϕ è vera per almeno un valore che la variabile x può assumere nell'universo dei valori.

Esempio 1 (Arity 1):
Uomo(socrat)
Esempio 2 (Arity 2):
Ama(romeo, giulietta)
Esempio 3 (Termini
Complessi):
Maggiore(successore(x), x)

Queste regole sono identiche a quelle della Logica Proposizionale



Interpretation

In FOL, il modello non solo assegna valori di verità, ma interpreta l'intero vocabolario non logico del linguaggio (Costanti, Funzioni e Predicati) all'interno di un universo specifico.

Structure of a model

Un Modello M (o Interpretazione) per un insieme di formule su un vocabolario (C, F, P) è una quadrupla $\langle A, C, F, P \rangle$ dove C, F, P sono le funzioni di interpretazione che si applicano all'universo dei valori A .

A model M for formulas over (C, F, P) is of the kind $\langle A, C, F, P \rangle$

- A is a non-empty universe of concrete values (the individuals)
- $C : C \mapsto A$ associates to each constant a concrete value
- $F : F \mapsto (A^n \mapsto A)$ associates to each function name a concrete function
- $P : P \mapsto (A^n \mapsto \{F, T\})$ associates to each predicate name a concrete predicate

Associa ad ogni costante un singolo valore in A .

Associa ad ogni nome di funzione n -aria f una funzione concreta da n individui in A ad un singolo individuo in A .

Associa ad ogni nome di predicato n -ario p un predicato concreto da n individui in A ad un valore di verità (Vero o Falso).

Understanding

That is, a specific model M associates:

- a term t (once its variables have been substituted with concrete values) to a concrete value
- an atomic formula $p(t_1, \dots, t_n)$ to a boolean value

Un termine t (una volta che le sue variabili sono state sostituite con valori concreti) produce un valore concreto.

Valutazione: Quando valutiamo un termine in un Modello M (es. padre(socrat), il risultato è sempre un individuo nel Dominio A .
Necessità di Assegnazione: Per valutare termini con variabili (es. padre(x)), è necessaria anche un'Assegnazione di Variabili, che specifica quale elemento di A corrisponde a x . Solo allora il termine si valuta in un valore concreto.

Valutazione: La formula atomica prende i valori concreti restituiti dai suoi termini $(t_1, \dots, t_n)$ e li inserisce nel predicato interpretato (P corsivo).
Risultato: Il risultato è la verità o falsità dell'affermazione nel Modello M .



An example of model: students and instructor

Formula

$$M \models (\forall X (stud(X) \rightarrow (\exists Y (instr(Y) \wedge younger(X, Y)))))$$

$$C = \{\}, F = \{\}, P = \{stud/1, instr/1, younger/2\}$$

La formula significa: "Per ogni individuo X, se X è uno studente, allora esiste almeno un individuo Y che è un istruttore ed è più giovane di Y."

$$\text{Model } M = \langle A, C, \mathcal{F}, \mathcal{P} \rangle$$

Poiché l'implicazione è vera per tutti gli individui nel dominio A, la formula quantificata universalmente è Vera nel Modello M

- $A = \{\text{bravetti}, \text{rossi}, \text{omicini}, \text{bianchi}, \text{natali}\}$ (i cinque individui)

- $C = \{\}, \mathcal{F} = \{\}$ Non ci sono costanti definite, quindi tutti gli individui sono riferiti tramite le variabili. Non ci sono funzioni definite

- $\mathcal{P} = \{$ L'interpretazione assegna (Vero) o (Falso) per ogni predicato applicato a elementi del dominio A.

- $stud(\text{bravetti})=F, stud(\text{rossi})=T, stud(\text{omicini})=F,$
 $stud(\text{bianchi})=T, stud(\text{natali})=F$

- $instr(\text{bravetti})=T, instr(\text{rossi})=F, instr(\text{omicini})=T,$
 $instr(\text{bianchi})=F, instr(\text{natali})=T$

- $younger(\text{rossi}, \text{natali})=T, younger(\text{natali}, \text{omicini})=F,$
 $younger(\text{bianchi}, \text{bravetti})=T, \dots$

Mi basta guardare se gli studenti sono younger dei professori (non mi serve guardare se i prof sono younger tra di loro)

}

i predicati vanno definiti su tutti i valori dell'universo (indip. dal valore: va fatto anche con se stesso)



An example of model: M_P , Peano arithmetics

Formulas

- Constant z
- Functions $s/1$, $+/2$, $*/2$ (in infix notation)
- Predicates $>/2$, $=/2$ (in infix notation)

The model $\langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$

- A : the set $\mathbb{N} = \{0, 1, 2, \dots\}$ of natural numbers
- $\mathcal{C}$ associates constant z to 0
- $\mathcal{F}$ associates function name $s/1$ to the function that computes the successor, $+/2$ to mathematical sum (e.g. $s(z) + s(z)$ is associated to 2), $\dots$
- $\mathcal{P}$ associates predicate name $>/2$ to the greater than predicate between numbers

Example of formulas

La relazione sussiste (Vero).

Giustificazione: Per soddisfare un quantificatore universale, la condizione deve valere per ogni x nell'universo dei valori A.

La relazione sussiste (Vero).

Giustificazione: Per soddisfare un quantificatore esistenziale (esiste y), basta trovare un singolo esempio nell'universo dei valori A.

Do these satisfaction relations hold?

Natural numbers are not closed under square root: I Numeri Naturali Non Sono Chiusi Sotto Radice Quadrata

- $M_P \models \exists y \neg \exists x (x * x = y)$ Traduzione Letterale: Esiste un numero y tale che non esiste nessun numero x il cui quadrato sia y. Significato in N: Questa formula afferma che esistono numeri naturali che non sono quadrati perfetti.

Existence of a greater value: Esistenza di un Valore Maggiore

- $M_P \models \forall x \exists y (y > x)$ Traduzione Letterale: Per ogni numero x, esiste un numero y tale che y è maggiore di x. Significato in N: Questa formula afferma che non esiste un numero naturale massimo; per ogni numero, ne esiste sempre uno più grande.

What is the following about?

- $M_P \models \forall q \exists p (p > q \wedge \forall x \forall y ((x > s(z) \wedge y > s(z)) \rightarrow \neg (x * y = p)))$

Per ogni numero q, esiste un numero p tale che p > q e p non può essere ottenuto come prodotto di due numeri entrambi maggiori di 1.

Un numero che non può essere ottenuto come prodotto di due numeri maggiori di 1 è, per definizione, un numero primo (o 0 o 1, ma la formula dice che p deve essere maggiore di q). Formula 3 Afferma: Per ogni numero q, esiste un numero primo p maggiore di q.

La relazione sussiste (Vero).

Teorema Logico: Il teorema afferma che, dato qualsiasi numero primo, ne esiste sempre uno più grande.

Giustificazione: Poiché i numeri primi non hanno una fine, dato qualsiasi q in N, è sempre possibile trovare un numero primo p tale che p > q.



Closed Formulas

Free and bound variables

- The occurrence of a variable x in formula ϕ is said to be *bound*, if that occurrence is inside a *binder* of kind $\forall x$ or $\exists x$
- The occurrence of a variable x in formula ϕ is said to be *free* if it is not *bound*
- Example: in $p(\boxed{x}) \wedge \forall x(p(\boxed{x}) \vee c)$ the left-most occurrence of x is free the right-most occurrence of x is bound

Un quantificatore lega tutte le occorrenze della variabile x nella sottoformula che segue. La variabile bound è interna, e il suo valore è specificato dal quantificatore.

Closed Formulas

ϕ is a closed formula if it does not include free variable occurrences

We will consider closed formulas only, unless stated otherwise.



L'intero processo è ben definito perché, partendo da una formula chiusa, ogni volta che si incontrano dei quantificatori (forall x, exists x), questi vengono eliminati tramite la sostituzione con valori del dominio (v), garantendo che alla fine si arrivi solo a formule atomiche che possono essere valutate come Vero o Falso nel modello M.

Semantics of predicate logic

La Semantica di FOL combina la Semantica della Logica Proposizionale con regole specifiche per i quantificatori e l'interpretazione dei termini.

Semantics (for closed formulas)

Definition of satisfaction relation $M \models \phi$

- $M \models p(t_1, \dots, t_n) \iff M(p(t_1, \dots, t_n)) = \mathbf{T}$

La formula atomica è vera nel modello M se e solo se la sua interpretazione (valutazione) nel dominio A è Vero.

- $M \models \top, M \not\models \perp$

- $M \models \neg\phi \iff M \not\models \phi$

- $M \models \phi \wedge \phi' \iff M \models \phi \text{ and } M \models \phi'$

- $M \models \phi \vee \phi' \iff M \models \phi \text{ or } M \models \phi'$

- $M \models \phi \rightarrow \phi' \iff M \models \phi \text{ implies } M \models \phi'$

- $M \models \forall x\phi \iff \text{for all } v \in A, \text{ we have } M \models \phi[v/x]$

Una formula forall x phi è vera nel modello M se e solo se la formula phi rimane vera quando la variabile libera x viene sostituita da ogni singolo individuo v nel Dominio A.

- $M \models \exists x\phi \iff \text{there exists } v \in A \text{ such that } M \models \phi[v/x]$

Una formula exists x phi è vera nel modello M se e solo se esiste almeno un individuo v nel Dominio A che, sostituito ad x in phi, rende la formula vera.

Quando si valuta un quantificatore, si considera un elemento v del dominio A.

where $\phi[v/x]$ means substituting *free occurrences* of x in ϕ with v

La definizione di $M \models \phi$ si basa sul controllo di $M \models \phi'$ per (sotto)formule phi' che includono valori v al posto di variabili libere x.

Note: definition of $M \models \phi$ is based on checking $M \models \phi'$ for (sub)formulas ϕ' including values v instead of free variables x

Le regole per i connettivi logici sono identiche a quelle della Logica Proposizionale, in quanto operano sui valori di verità delle sottoformule



Semantics of predicate logic

La valutazione $M(\dots)$ di un elemento nel linguaggio della Logica del Predicato produce:
Valori di A (se l'elemento è un Termine t).
Valori Booleani (se l'elemento è una Formula Atomica).

Semantics: formal definition of $M(p(t_1, \dots, t_n))$

Remember: model $M = \langle A, \mathcal{C}, \mathcal{F}, \mathcal{P} \rangle$

Restituisce un valore di verità (Vero o Falso)

$M()$ over atomic formulas

Prima si valutano tutti i termini t_i nei rispettivi valori di A . Poi, questi individui sono inseriti nel predicato concreto $\mathcal{P}_{\text{corsivo}}(p)$ associato al simbolo p . Il risultato è Vero o Falso.

- $M(p(t_1, \dots, t_n)) = \mathcal{P}(p)(M(t_1), \dots, M(t_n))$

Una Formula Atomica $p(t_1, \dots, t_n)$ viene valutata nel Modello M in un valore booleano.

Restituisce un valori dell'universo dei valori di A

$M()$ over terms t (with variables x substituted by values v)

- $M(v) = v$ La variabile, una volta assegnata a un valore concreto v dell'universo di valori A , si valuta in quello stesso valore v .

- $M(c) = \mathcal{C}(c)$ Il simbolo di costante viene valutato nel valore specifico a cui è associato dalla funzione di interpretazione $\mathcal{C}_{\text{corsivo}}$.

- $M(f(t_1, \dots, t_n)) = \mathcal{F}(f)(M(t_1), \dots, M(t_n))$

Si valuta ricorsivamente ogni sotto-termini t_i in valori di A . Questi valori sono poi inseriti nella funzione concreta $\mathcal{F}_{\text{corsivo}}(f)$ associata al simbolo di funzione f . Il risultato è un singolo valore in A .

quando arrivo a valutare i predicati, ho già sostituito le variabili con valori dell'universo di valori con i quantificatori universali ed esistenziali



Logical Consequence/Equivalence, Validity/Satisfiability

Same definitions as for Propositional Logic

Example of logical equivalence

- $\exists x\phi \equiv \neg\forall x\neg\phi$

A simplified version of Predicate Logic

Syntax: $\phi ::= p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \rightarrow \phi \mid \forall x\phi$

(Semantics is the simplified version of what we saw before)



Validity in predicate logic is undecidable!!

Non esiste un algoritmo in grado di prendere in input una formula ϕ arbitraria della Logica del Predicato e restituire, in un tempo finito garantito, una risposta definitiva (SÌ o NO) alla domanda: "È ϕ una Tautologia ($\models \phi$)?".

Undecidability of the validity problem for predicate logic

Turing and Church independently proved that:

Il problema $\models \phi$ è in generale indecidibile (su formule ϕ che usano l'uguaglianza ed un altro predicato con numero di parametri di almeno 2).

The problem $\models \phi$ is in general undecidable (over formulas ϕ using equality and another predicate with arity at least 2)

Sebbene la validità non sia decidibile (non esiste un algoritmo che dica sempre SÌ o NO in tempo finito), il problema è semi-decidibile (ricorsivamente enumerabile).

Semi-decidability of the validity problem for predicate logic

Esiste una procedura che implementa $\models \phi$, per una ϕ generale, tale che: Se $\models \phi$, viene fornita una risposta positiva in una quantità finita di tempo. Se non soddisfa ϕ , la procedura non termina.

A procedure implementing $\models \phi$, for a general ϕ , exists such that:

- if $\models \phi$, a positive answer is provided in a finite amount of time
- if $\not\models \phi$, the procedure does not terminate

Questa proprietà deriva dal Teorema di Completezza di Gödel per FOL, che stabilisce che la validità ($\models$) e la dimostrabilità ($\vdash$) sono equivalenti.

- Se $\models \phi$ (è valida): Per il Teorema di Completezza, sappiamo che esiste una dimostrazione formale $\vdash \phi$. Possiamo avviare un sistema di deduzione per cercare questa dimostrazione. Poiché la dimostrazione è una sequenza finita di passi, la procedura troverà e fornirà la dimostrazione in tempo finito.

- Se non $\models \phi$ (non è valida): Non esiste alcuna dimostrazione. Se facessimo cercare all'algoritmo una dimostrazione, esso cercherebbe all'infinito senza mai trovarla e, non essendoci un limite superiore finito alla lunghezza della dimostrazione, non possiamo mai sapere se la ricerca fallirà o se la dimostrazione è solo più lunga. L'algoritmo non terminerà.



Necessità: A differenza della Semantica ($\models$), dove si preferiscono formule chiuse, i Sistemi di Deduzione Sintattica ($\vdash$) devono lavorare con formule aperte (con variabili libere) nei passaggi intermedi.
 Motivo: Le regole come UG (Generalizzazione Universale) richiedono che le variabili x siano trattate come generiche o specializzate prima di essere quantificate o usate in una dimostrazione.
 Focus Finale: Anche se si usano formule aperte, l'obiettivo finale della dimostrazione è spesso raggiungere una formula chiusa o una derivazione $\phi_1, \dots, \phi_n \vdash \phi$ in cui ϕ è chiuso (o non introduce variabili free).

Hilbert deduction system

Mentre gli assiomi e le regole per la Logica Proposizionale gestiscono i connettivi logici, gli assiomi aggiuntivi e la regola di Generalizzazione Universale gestiscono i quantificatori.

Deduction system

Minimal syntax: $\phi ::= p(t_1, \dots, t_n) \mid \neg\phi \mid \phi \rightarrow \phi \mid \forall x\phi$

Axioms:

- .. those of Propositional Logic.. and

- $\vdash (\forall x\alpha) \rightarrow \alpha[t/x]$ with no free variable in t becoming bound

- $\vdash \forall x(\alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \forall x\beta)$ with x not free in α

Rules:

- $$\frac{\vdash \alpha \quad \vdash \alpha \rightarrow \beta}{\vdash \beta} \quad (\text{Modus Ponens})$$

- $$\frac{\vdash \phi}{\vdash \forall x\phi} \quad (\text{Universal Generalisation})$$

where $\phi[t/x]$ means substituting *free occurrences* of x in ϕ with t

Se t contiene una variabile libera y , e quella y viene legata (diventa bound) da un quantificatore esistente in α durante la sostituzione, il significato logico sarebbe alterato.

Se x fosse variabile free in α , non potremmo "muovere" il quantificatore universale in quel modo, poiché il valore di verità di α varierebbe con x . Quindi, se α non dipende da x , e l'implicazione $\alpha \rightarrow \beta$ vale per tutti gli x , allora è la stessa cosa che dire che α implica che β è universalmente vero.

Se una formula ϕ è dimostrabile senza fare affidamento su alcuna assunzione sulla variabile x (cioè x è trattata come generica), allora possiamo concludere che ϕ vale per tutti gli x .

esempio

E.g. last rule can be used to derive $\forall x p(x) \rightarrow (q(x) \rightarrow p(x))$
 (in *intermediate* steps of derivations works with open formulas)



Il sistema di deduzione è un ponte perfetto tra la sintassi (dimostrazione) e la semantica (validità).

- Correttezza (Soundness): Se $\vdash \phi$ allora $\models \phi$.

> Significato: Tutto ciò che può essere dimostrato sintatticamente (tramite gli assiomi e le regole di Hilbert) è logicamente valido (vero in tutti i modelli). Il sistema non produce errori.

- Completezza (Completeness): Se $\models \phi$ allora $\vdash \phi$.

> Significato: Tutto ciò che è logicamente valido (vero in tutti i modelli) può essere dimostrato sintatticamente nel sistema di Hilbert. Il sistema è sufficientemente potente da catturare ogni verità logica.

(Una formula è valida se e solo se ha una prova.)

Questi due risultati combinati, noti come il Teorema di Completezza di Gödel, stabiliscono che i concetti di validità ($\models$) e dimostrabilità ($\vdash$) sono equivalenti per la Logica del Predicato (Primo Ordine).

Correctness and completeness

It can be proved that $\vdash$ is still correct and complete (Gödel)
(a formula is valid if and only if it has a proof)

Undecidability

$\vdash$ is undecidable
(consequence of undecidability of $\models$)

Nonostante il successo del Teorema di Completezza, il problema di stabilire la validità rimane insolubile per un algoritmo generale.

- Indecidibilità di $\vdash$:

> Significato: Non esiste un algoritmo che possa decidere, in un tempo finito garantito, se una formula ϕ è dimostrabile ($\vdash \phi$).

> Conseguenza: Questo è una diretta conseguenza dell'Indecidibilità della Validità ($\models \phi$), come stabilito da Church e Turing.

Il Paradosso Computazionale

- Il sistema di deduzione di Hilbert in FOL è quindi:

- Perfetto: Se una verità esiste (Completezza), il sistema può dimostrarla.

- Inaffidabile (computazionalmente): Non esiste un metodo generale per sapere se una formula non è vera (non è valida o dimostrabile) in tempo finito, perché l'algoritmo potrebbe non terminare la sua ricerca di una dimostrazione (Semi-Decidibilità).

In sintesi, la Logica del Predicato è Completa e Corretta (teoricamente potente) ma Indecidibile (computazionalmente difficile).

